

LAPORAN PRATIUM
STRUKTUR DATA
“Sorting dan GUI InsertionSorting”



Dosen Pengampu:
DR. Wahyudi. .S.T.M.T

Asisten Lab:
Jovantri Immanuel Gulo

Disusun Oleh:
Marcello Bayu Denar Widjaksono
2511532011

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum mata kuliah Struktur Data, dengan judul: “Sorting dan GUI InsertionSorting”.

Pada kesempatan ini, Penulis mengucapkan terima kasih Bapak DR. Wahyudi. .S.T.M.T yang telah memberikan tugas laporan pratikum. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari sempurna dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis mengharapkan segala bentuk saran dan masukan serta kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap Laporan ini dapat memberikan manfaat bagi perkembangan dunia pendidikan.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam ilmu komputer dan struktur data, pengurutan (sorting) merupakan salah satu operasi fundamental yang paling sering digunakan untuk mengorganisir data agar lebih mudah dicari, dianalisis, atau diproses lebih lanjut. Terdapat berbagai macam algoritma pengurutan dengan karakteristik kompleksitas waktu dan ruang yang berbeda-beda, seperti Insertion Sort, Selection Sort, dan Bubble Sort. Pemahaman mendalam mengenai logika kerja masing-masing algoritma sangat penting bagi mahasiswa informatika untuk dapat memilih metode yang paling efisien sesuai dengan konteks permasalahan.

Selain implementasi algoritma berbasis konsol, visualisasi proses pengurutan melalui Graphical User Interface (GUI) menjadi metode yang efektif untuk memahami bagaimana data berpindah dan dibandingkan secara step-by-step. Melalui praktikum ini, mahasiswa tidak hanya dituntut untuk menulis kode pengurutan yang fungsional, tetapi juga membangun aplikasi interaktif menggunakan Java Swing untuk memvisualisasikan algoritma Insertion Sort. Hal ini bertujuan untuk menjembatani pemahaman teoritis dengan implementasi nyata serta meningkatkan kemampuan pemrograman visual.

1.2 Tujuan Pratikum

1. Mengimplementasikan algoritma pengurutan Insertion Sort, Selection Sort, dan Bubble Sort menggunakan bahasa pemrograman Java.
2. Memahami logika perbandingan dan pertukaran elemen pada masing-masing algoritma pengurutan.
3. Membangun aplikasi GUI menggunakan Java Swing untuk memvisualisasikan proses Insertion Sort secara langkah demi langkah.
4. Menganalisis perubahan array sebelum dan setelah proses pengurutan dilakukan.

1.3 Manfaat Pratikum

1. Mahasiswa mampu mengimplementasikan algoritma sorting dasar secara mandiri dalam bentuk kode program.
2. Mahasiswa memahami mekanisme in-place sorting dimana pengurutan dilakukan tanpa memerlukan array tambahan yang signifikan.
3. Mahasiswa mendapatkan pengalaman praktis dalam mengembangkan antarmuka grafis (GUI) menggunakan komponen Swing seperti JFrame, JPanel, JButton, dan JLabel.
4. Visualisasi GUI membantu mahasiswa untuk melihat secara nyata bagaimana elemen digeser dan disisipkan pada algoritma Insertion Sort, sehingga memperkuat pemahaman konseptual.

BAB II PEMBAHASAN

2.1 Pratikum Striktur Data

```
package pekan7_2511532011;

public class InsertionSort_2511532011 {
    public static void InsertionSort_2511532011(int[] arr_2011) {
        int n_2011 = arr_2011.length;
        for ( int i_2011 = 1; i_2011 < n_2011; i_2011++) {
            int key_2011 = arr_2011[i_2011];
            int j_2011 = i_2011 - 1 ;
            while (j_2011 >= 0 && arr_2011[j_2011] > key_2011 )
            {
                arr_2011[j_2011 + 1] = arr_2011[j_2011];
                j_2011 --;
            }
            arr_2011[j_2011 + 1] = key_2011;
        }
    }

    public static void main( String[] args) {
        int arr_2011[] = {23, 78, 45, 8, 32, 56, 1};
        int n_2011 = arr_2011.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i = 0; i < n_2011; i++) {
            System.out.print(arr_2011[i] + " ");
        }
        System.out.println("");
        InsertionSort_2511532011(arr_2011);
        System.out.printf("array yang sudah terurut:\n");
        for (int i = 0; i < n_2011; i++) {
            System.out.print(arr_2011[i] + " ");
        }
        System.out.println("");
    }
}
```

Pembahasan Class InsertionSort:

1. Deklarasi Package dan Kelas

package pekan7_2511532011;: Menetapkan namespace kelas agar terorganisir dalam direktori praktikum minggu ke-7 yang membahas algoritma pengurutan.

public class InsertionSort_2511532011: Kelas publik yang berfungsi sebagai wadah implementasi algoritma dan driver pengujian.

2. Metode InsertionSort_2511532011 (Inti Algoritma)

public static void InsertionSort_2511532011(int[] arr_2011): Metode statis yang menerima array integer sebagai parameter dan melakukan pengurutan secara in-place (tanpa array tambahan).

int n_2011 = arr_2011.length;: Menyimpan panjang array ke variabel n_2011 untuk menghindari pemanggilan berulang metode .length di dalam loop.

for (int i_2011 = 1; i_2011 < n_2011; i_2011++): Outer Loop. Iterasi dimulai dari index 1 karena elemen pertama (index 0) dianggap sudah terurut secara trivial. Loop berjalan hingga akhir array.

int key_2011 = arr_2011[i_2011];: Menyimpan elemen saat ini yang akan disisipkan ke posisi yang benar di bagian kiri array yang sudah terurut. Elemen ini disebut key.

int j_2011 = i_2011 - 1;: Inisialisasi pointer j yang menunjuk ke elemen tepat sebelum key. Pointer ini akan bergerak mundur untuk membandingkan dan menggeser elemen.

while (j_2011 >= 0 && arr_2011[j_2011] > key_2011): Inner Loop (Shift Logic). Berjalan selama j masih valid (tidak negatif) dan elemen di posisi j lebih besar dari key. Kondisi ini berarti key harus disisipkan lebih ke kiri.

arr_2011[j_2011 + 1] = arr_2011[j_2011];: Menggeser elemen yang lebih besar dari key satu posisi ke kanan untuk membuat ruang kosong.

j_2011--;: Menggeser pointer j satu langkah ke kiri untuk melanjutkan perbandingan dengan elemen sebelumnya.

arr_2011[j_2011 + 1] = key_2011;: Insertion Step. Setelah loop while berhenti, j+1 adalah posisi yang tepat untuk key. Nilai key disisipkan ke posisi tersebut.

Metode main - Inisialisasi Data

int arr_2011[] = {23, 78, 45, 8, 32, 56, 1};: Mendeklarasikan dan menginisialisasi array integer dengan 7 elemen acak sebagai data uji.

int n_2011 = arr_2011.length;: Menyimpan panjang array untuk digunakan dalam perulangan pencetakan.

3. Pencetakan Sebelum Pengurutan

System.out.printf("array yang belum terurut:\n");: Mencetak label output.

```
for (int i = 0; i < n_2011; i++) { System.out.print(arr_2011[i] + " "); }:
```

Iterasi array dan mencetak setiap elemen dipisahkan spasi untuk menampilkan kondisi awal.

4. Pemanggilan Metode Pengurutan

`InsertionSort_2511532011(arr_2011);`: Memanggil metode statis sorting. Karena Java melewati referensi array secara by-reference, perubahan yang terjadi di dalam metode akan langsung mempengaruhi array `arr_2011` di main.

5. Pencetakan Setelah Pengurutan

`System.out.printf("array yang sudah terurut:\n");`: Mencetak label output hasil.

```
for (int i = 0; i < n_2011; i++) { System.out.print(arr_2011[i] + " "); }:
```

Mencetak array yang telah terurut secara menaik (ascending).


```

package pekan7_2511532011;

public class SelectionSort_2511532011 {
    public static void SelectionSort_2511532011(int[] arr_2011) {
        int n_2011 = arr_2011.length;
        for ( int i_2011 = 0; i_2011 < n_2011; i_2011++) {
            int minIndex_2011 = i_2011;
            for ( int j_2011 = i_2011 + 1; j_2011 < n_2011; j_2011++)
            {
                if ( arr_2011[j_2011] < arr_2011[minIndex_2011] ) {
                    minIndex_2011 = j_2011;
                }
            }
            int temp_2011 = arr_2011[i_2011];
            arr_2011[i_2011] = arr_2011[minIndex_2011];
            arr_2011[minIndex_2011] = temp_2011;
        }
    }

    public static void main( String[] args ) {
        int arr_2011[] = {23, 78, 45, 8, 32, 56, 1};
        int n_2011 = arr_2011.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i = 0; i < n_2011; i++) {
            System.out.print(arr_2011[i] + " ");
        }
        System.out.println("");
        SelectionSort_2511532011(arr_2011);
        System.out.printf("array yang sudah terurut:\n");
        for (int i = 0; i < n_2011; i++) {
            System.out.print(arr_2011[i] + " ");
        }
        System.out.println("");
    }
}

```

Pembahasan Class SelectionSort:

1. Deklarasi Package dan Kelas

package pekan7_2511532011;: Menetapkan namespace kelas agar terorganisir dalam direktori praktikum minggu ke-7 yang fokus pada algoritma pengurutan (sorting).

public class SelectionSort_2511532011: Kelas publik yang berfungsi sebagai wadah implementasi algoritma Selection Sort dan driver pengujian.

2. Metode SelectionSort_2511532011 (Inti Algoritma)

public static void SelectionSort_2511532011(int[] arr_2011): Metode statis yang menerima array integer dan melakukan pengurutan secara in-place (tanpa array tambahan).

int n_2011 = arr_2011.length;; Menyimpan panjang array ke variabel lokal untuk efisiensi akses di dalam perulangan.

for (int i_2011 = 0; i_2011 < n_2011; i_2011++): Outer Loop. Berfungsi untuk menentukan posisi target tempat elemen minimum akan diletakkan. Iterasi dimulai dari index 0 hingga akhir array.

int minIndex_2011 = i_2011;; Asumsi awal bahwa elemen pada posisi i adalah elemen terkecil di bagian array yang belum terurut.

for (int j_2011 = i_2011 + 1; j_2011 < n_2011; j_2011++): Inner Loop. Memindai sisa elemen di sebelah kanan posisi i untuk mencari nilai yang lebih kecil.

if (arr_2011[j_2011] < arr_2011[minIndex_2011]) { minIndex_2011 = j_2011; }; Logika Pencarian Minimum. Jika ditemukan elemen yang lebih kecil dari elemen yang saat ini dianggap minimum, update minIndex_2011 ke index elemen baru tersebut.

int temp_2011 = arr_2011[i_2011]; arr_2011[i_2011] = arr_2011[minIndex_2011]; arr_2011[minIndex_2011] = temp_2011;;

Logika Swap. Menukar elemen di posisi i dengan elemen minimum yang ditemukan. Setelah swap, elemen di posisi i dijamin terurut dengan benar.

Catatan Optimasi: Loop i sebenarnya bisa berhenti di n_2011 - 1 karena elemen terakhir akan otomatis berada di posisi yang tepat setelah n-1 iterasi. Kode saat ini tetap berjalan dengan benar namun melakukan satu iterasi tambahan yang tidak mengubah hasil.

3. Metode main - Inisialisasi Data

int arr_2011[] = {23, 78, 45, 8, 32, 56, 1};; Mendeklarasikan array dengan 7 elemen acak sebagai data uji.

int n_2011 = arr_2011.length;; Menyimpan panjang array untuk kontrol perulangan pencetakan.

Pencetakan Sebelum Pengurutan

System.out.printf("array yang belum terurut:\n");; Label output kondisi awal.

for (int i = 0; i < n_2011; i++) { System.out.print(arr_2011[i] + " "); };

Iterasi dan cetak elemen array dipisahkan spasi.

Pemanggilan Metode Pengurutan

`SelectionSort_2511532011(arr_2011);` Memanggil algoritma. Karena array dilewatkan secara by-reference di Java, perubahan yang terjadi di dalam metode langsung memodifikasi array asli di main.

4. Pencetakan Setelah Pengurutan

`System.out.printf("array yang sudah terurut:\n");` Label output hasil.

`for (int i = 0; i < n_2011; i++) { System.out.print(arr_2011[i] + " "); }`

Mencetak array yang telah terurut secara menaik (ascending).

```

package pekan7_2511532011;

public class BubleSort_2011 {
    public static void BubleSort_2511532011(int[] arr_2011) {
        int n_2011 = arr_2011.length;
        for ( int i_2011 = 0; i_2011 < n_2011 ; i_2011++) {
            for ( int j_2011 = 0; j_2011 < n_2011 - i_2011 - 1; j_2011++) {
                if (arr_2011[j_2011] > arr_2011[j_2011 + 1 ]) {
                    int temp_2011 = arr_2011[j_2011];
                    arr_2011[j_2011] = arr_2011[j_2011 + 1];
                    arr_2011[j_2011 + 1] = temp_2011;
                }
            }
        }
    }

    public static void main( String[] args) {
        int arr_2011[] = {23, 76, 45, 8, 32, 56, 1};
        int n_2011 = arr_2011.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i = 0; i < n_2011; i++) {
            System.out.print(arr_2011[i] + " ");
        }
        System.out.println("");
        BubleSort_2511532011(arr_2011);
        System.out.printf("array yang sudah terurut:\n");
        for (int i = 0; i < n_2011; i++) {
            System.out.print(arr_2011[i] + " ");
        }
        System.out.println("");
    }
}

```

Pembahasan Class BubleSort:

1. Deklarasi Package dan Kelas

package pekan7_2511532011;: Menetapkan namespace kelas agar terorganisir dalam direktori praktikum minggu ke-7 yang membahas algoritma pengurutan.

public class BubleSort_2011: Kelas publik yang berfungsi sebagai wadah implementasi algoritma Bubble Sort dan driver pengujian. (Catatan: terdapat typo pada penamaan kelas "Buble", seharusnya "Bubble", namun tidak mengganggu eksekusi program).

2. Metode BubleSort_2511532011 (Inti Algoritma)

public static void BubleSort_2511532011(int[] arr_2011): Metode statis yang menerima array integer dan melakukan pengurutan secara in-place (tanpa array tambahan).

`int n_2011 = arr_2011.length;` Menyimpan panjang array ke variabel lokal untuk menghindari pemanggilan `.length` berulang di dalam loop.

Outer Loop (Pengaturan Pass)

`for (int i_2011 = 0; i_2011 < n_2011 ; i_2011++):` Loop ini menentukan jumlah pass (lewat) pengurutan. Setiap iterasi `i` menjamin bahwa satu elemen terbesar akan "mengggelembung" ke posisi paling akhir yang benar.

Inner Loop (Perbandingan Bersebelahan)

`for (int j_2011 = 0; j_2011 < n_2011 - i_2011 - 1; j_2011++):` Loop yang melakukan perbandingan elemen bersebelahan.

Optimasi `n_2011 - i_2011 - 1`: Ini adalah bagian kritis. Setelah pass ke-`i`, sebanyak `i` elemen terakhir sudah berada di posisi terurut yang benar. Batas ini mencegah algoritma membandingkan ulang elemen yang sudah fix, sehingga mengurangi iterasi yang tidak perlu.

Kondisi Pertukaran (Swap Condition)

`if (arr_2011[j_2011] > arr_2011[j_2011 + 1 ]):` Membandingkan elemen di index `j` dengan elemen tepat di sebelah kanannya (`j+1`). Jika elemen kiri lebih besar, berarti urutan salah untuk pengurutan menaik (ascending), sehingga perlu ditukar.

Logika Swap (3 Langkah)

`int temp_2011 = arr_2011[j_2011];` Simpan nilai elemen kiri ke variabel temporary.

`arr_2011[j_2011] = arr_2011[j_2011 + 1];` Timpa posisi kiri dengan nilai elemen kanan.

`arr_2011[j_2011 + 1] = temp_2011;` Isi posisi kanan dengan nilai yang disimpan di temporary. Kedua elemen kini bertukar posisi.

3. Metode main - Inisialisasi Data

`int arr_2011[] = {23, 76, 45, 8, 32, 56, 1};` Mendeklarasikan dan mengisi array dengan 7 elemen acak sebagai data uji.

`int n_2011 = arr_2011.length;` Menyimpan panjang array untuk kontrol perulangan pencetakan.

Pencetakan Sebelum Pengurutan

System.out.printf("array yang belum terurut:\n");: Mencetak label output kondisi awal.

```
for (int i = 0; i < n_2011; i++) { System.out.print(arr_2011[i] + " "); }:
```

Iterasi dan mencetak setiap elemen dipisahkan spasi.

4. Pemanggilan Metode Pengurutan

BubleSort_2511532011(arr_2011);: Memanggil algoritma sorting. Karena Java melewatkan array secara by-reference, modifikasi yang terjadi di dalam metode langsung mengubah array arr_2011 di main.

Pencetakan Setelah Pengurutan

System.out.printf("array yang sudah terurut:\n");: Mencetak label output hasil.

```
for (int i = 0; i < n_2011; i++) { System.out.print(arr_2011[i] + " "); }:
```

Mencetak array yang telah

```

package pekan7_2511532011;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;

import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.border.EmptyBorder;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JTextArea;
import java.awt.Font;

public class InsertionSortGUI_2511532011 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_2011;
    private JLabel [] labelArray_2011;
    private JButton stepButton_2011, resetButton_2011, setButton_2011;
    private JTextField inputField_2011;
    private JPanel panelArray_2011;
    private JTextArea stepArea_2011;

    private int i_2011 = 1, j_2011;
    private boolean sorting_2011 = false;
    private int stepCount_2011 = 1;

    /**
     * Create the frame.
     */
    public InsertionSortGUI_2511532011() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2011 = new JPanel(new FlowLayout());
        inputField_2011 = new JTextField(30);
        setButton_2011 = new JButton("Set Array");
        inputPanel_2011.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
        inputPanel_2011.add(inputField_2011);
        inputPanel_2011.add(setButton_2011);

        // Panel array visual
        panelArray_2011 = new JPanel();
        panelArray_2011.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_2011 = new JPanel();
        stepButton_2011 = new JButton("Langkah Selanjutnya");
        resetButton_2011 = new JButton("Reset");
        stepButton_2011.setEnabled(false);
        controlPanel_2011.add(stepButton_2011);
        controlPanel_2011.add(resetButton_2011);

        // Area teks untuk log langkah-langkah
        stepArea_2011 = new JTextArea(8, 60);
        stepArea_2011.setEditable(false);
        stepArea_2011.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane_2011 = new JScrollPane(stepArea_2011);

        // Tambahkan panel ke frame
        add(inputPanel_2011, BorderLayout.NORTH);

```

```

add(panelArray_2011, BorderLayout.CENTER);
add(controlPanel_2011, BorderLayout.SOUTH);
add(scrollPane_2011, BorderLayout.EAST);

// Event Set Array
setButton_2011.addActionListener(e_2011 -> setArrayFromInput_2011());

// Event Langkah Selanjutnya
stepButton_2011.addActionListener(e_2011 -> performStep_2011());

// Event Reset
resetButton_2011.addActionListener(e_2011 -> reset_2011());
}

private void setArrayFromInput_2011() {
    String text_2011 = inputField_2011.getText().trim();
    if (text_2011.isEmpty()) return;
    String[] parts_2011 = text_2011.split(",");
    array_2011 = new int[parts_2011.length];
    try {
        for (int k_2011 = 0; k_2011 < parts_2011.length; k_2011++) {
            array_2011[k_2011] = Integer.parseInt(parts_2011[k_2011].trim());
        }
    } catch (NumberFormatException e_2011) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
            + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    i_2011 = 1;
    stepCount_2011 = 1;
    sorting_2011 = true;
    stepButton_2011.setEnabled(true);
    stepArea_2011.setText("");
    panelArray_2011.removeAll();
    labelArray_2011 = new JLabel[array_2011.length];

    for (int k_2011 = 0; k_2011 < array_2011.length; k_2011++) {
        labelArray_2011[k_2011] = new JLabel(String.valueOf(array_2011[k_2011]));
        labelArray_2011[k_2011].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_2011[k_2011].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_2011[k_2011].setPreferredSize(new Dimension(50, 50));
        labelArray_2011[k_2011].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_2011.add(labelArray_2011[k_2011]);
    }

    panelArray_2011.revalidate();
    panelArray_2011.repaint();
}

private void performStep_2011() {
    if ( i_2011 < array_2011.length && sorting_2011 ) {
        int key_2011 = array_2011[i_2011];
        j_2011 = i_2011 - 1;

        StringBuilder stepLog_2011 = new StringBuilder();
        stepLog_2011.append("Langkah ").append(key_2011).append("\n");

        while (j_2011 >= 0 && array_2011[j_2011] > key_2011) {
            array_2011[j_2011 + 1] = array_2011[j_2011];
            j_2011--;
        }
        array_2011[j_2011 + 1] = key_2011;

        updateLabels_2011();
        stepLog_2011.append("Hasil: ").append(arrayToString_2011(array_2011)).append("\n\n");
        stepArea_2011.append(stepLog_2011.toString());

        i_2011++;
        stepCount_2011++;

        if ( i_2011 == array_2011.length ) {
            sorting_2011 = false;
            stepButton_2011.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting Selesai!");
        }
    }
}

private void updateLabels_2011() {
    for (int k_2011 = 0; k_2011 < array_2011.length; k_2011++) {
        labelArray_2011[k_2011].setText(String.valueOf(array_2011[k_2011]));
    }
}

private void reset_2011() {
    inputField_2011.setText("");
    panelArray_2011.removeAll();
    panelArray_2011.revalidate();
    panelArray_2011.repaint();
    stepArea_2011.setText("");
    stepButton_2011.setEnabled(false);
    sorting_2011 = false;
    i_2011 = 1;
}

```



```

        stepCount_2011 = 1;
    }

    private String arrayToString_2011(int[] arr_2011) {
        StringBuilder sb_2011 = new StringBuilder();
        for (int k_2011 = 0; k_2011 < arr_2011.length; k_2011++) {
            sb_2011.append(arr_2011[k_2011]);
            if (k_2011 < arr_2011.length - 1) sb_2011.append(", ");
        }
        return sb_2011.toString();
    }

    public static void main(String[] args_2011) {
        SwingUtilities.invokeLater(() -> {
            InsertionSortGUI_2511532011 gui_2011 = new InsertionSortGUI_2511532011();
            gui_2011.setVisible(true);
        });
    }
}

```

Pembahasan Class InsertionSortGui:

Deklarasi Package dan Import Library Swing

package pekan7_2511532011;; Menetapkan namespace kelas dalam direktori praktikum minggu ke-7.

Import java.awt.* dan javax.swing.*: Mengimpor komponen GUI Java seperti JFrame, JPanel, JButton, JLabel, JTextArea, dan layout manager (BorderLayout, FlowLayout) untuk membangun antarmuka pengguna.

Import java.awt.EventQueue dan SwingUtilities: Penting untuk thread safety dalam aplikasi Swing.

Deklarasi Kelas dan Inheritance

public class InsertionSortGUI_2511532011 extends JFrame: Kelas ini mewarisi (extends) JFrame, sehingga instance kelas ini langsung merupakan sebuah jendela aplikasi GUI.

private static final long serialVersionUID = 1L;; Identifier versi untuk serialisasi objek, praktik standar dalam kelas Swing.

Deklarasi Variabel Instance (State Management)

private int[] array_2011;; Array integer yang menyimpan data yang akan diurutkan.

private JLabel[] labelArray_2011;; Array referensi ke komponen JLabel untuk menampilkan setiap elemen array secara visual.

private JButton stepButton_2011, resetButton_2011, setButton_2011;; Referensi ke tombol-tombol kontrol untuk interaksi pengguna.

private JTextField inputField_2011;; Komponen input teks untuk memasukkan data array.

private JPanel panelArray_2011;; Panel kontainer untuk menampilkan visualisasi array (label-label angka).

private JTextArea stepArea_2011;; Area teks multi-line untuk menampilkan log/catatan setiap langkah algoritma.

private int i_2011 = 1, j_2011;; Variabel penanda posisi dalam algoritma Insertion Sort (i untuk elemen kunci, j untuk perbandingan).

private boolean sorting_2011 = false;; Flag status untuk mengontrol apakah proses sorting sedang aktif.

private int stepCount_2011 = 1;; Counter untuk penomoran langkah dalam log.

Konstruktor - Konfigurasi Jendela Utama

setTitle("Insertion Sort Langkah per Langkah"); Menetapkan judul jendela aplikasi.

setSize(750, 400); Menetapkan dimensi awal jendela (lebar 750px, tinggi 400px).

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); Menjamin aplikasi benar-benar keluar (terminasi JVM) saat jendela ditutup.

setLocationRelativeTo(null); Menempatkan jendela di tengah layar saat pertama kali muncul.

setLayout(new BorderLayout()); Menggunakan layout manager BorderLayout yang membagi area menjadi 5 zona: NORTH, SOUTH, EAST, WEST, CENTER.

Konstruktor - Panel Input (NORTH)

JPanel inputPanel_2011 = new JPanel(new FlowLayout()); Membuat panel dengan FlowLayout untuk menata komponen secara horizontal berurutan.

inputField_2011 = new JTextField(30); Membuat field teks dengan lebar 30 kolom untuk input pengguna.

setButton_2011 = new JButton("Set Array"); Membuat tombol untuk memproses input.

Komponen ditambahkan ke panel dan panel ditambahkan ke frame dengan add(inputPanel_2011, BorderLayout.NORTH);.

Konstruktor - Panel Visualisasi Array (CENTER)

panelArray_2011 = new JPanel(); Panel kosong yang akan diisi dinamis dengan JLabel sesuai jumlah elemen array.

panelArray_2011.setLayout(new FlowLayout()); Menggunakan FlowLayout agar label-label angka tersusun rapi secara horizontal.

`add(panelArray_2011, BorderLayout.CENTER);` Menempatkan panel visualisasi di area tengah (zona terbesar) jendela.

Konstruktor - Panel Kontrol (SOUTH)

`JPanel controlPanel_2011 = new JPanel();` Panel untuk menampung tombol-tombol aksi.

`stepButton_2011 = new JButton("Langkah Selanjutnya");` Tombol untuk mengeksekusi satu iterasi algoritma.

`resetButton_2011 = new JButton("Reset");` Tombol untuk mengulang/mengosongkan aplikasi.

`stepButton_2011.setEnabled(false);` Penting: Tombol step dinonaktifkan awal karena belum ada data array yang diproses.

`add(controlPanel_2011, BorderLayout.SOUTH);` Menempatkan panel kontrol di bagian bawah jendela.

Konstruktor - Area Log Langkah (EAST)

`stepArea_2011 = new JTextArea(8, 60);` Membuat area teks dengan 8 baris dan 60 kolom untuk menampilkan log algoritma.

`stepArea_2011.setEditable(false);` Membuat area teks hanya bisa dibaca (read-only) agar pengguna tidak bisa mengedit log.

`stepArea_2011.setFont(new Font("Monospaced", Font.PLAIN, 14));` Menggunakan font monospaced agar teks log rata dan mudah dibaca.

`JScrollPane scrollPane_2011 = new JScrollPane(stepArea_2011);` Membungkus `JTextArea` dengan `JScrollPane` agar muncul scrollbar jika konten melebihi area tampil.

`add(scrollPane_2011, BorderLayout.EAST);` Menempatkan area log di sisi kanan jendela.

Konstruktor - Event Handling dengan Lambda Expression

`setButton_2011.addActionListener(e_2011 -> setArrayFromInput_2011());` Mendaftarkan listener yang memanggil metode `setArrayFromInput_2011()` saat tombol "Set Array" diklik. Menggunakan lambda expression (`e_2011 ->`) sebagai sintaks ringkas untuk interface `ActionListener`.

stepButton_2011.addActionListener(e_2011 -> performStep_2011());: Listener untuk tombol "Langkah Selanjutnya" yang memicu eksekusi satu langkah algoritma.

resetButton_2011.addActionListener(e_2011 -> reset_2011());: Listener untuk tombol "Reset" yang mengembalikan aplikasi ke state awal.

Metode setArrayFromInput_2011() - Parsing Input

String text_2011 = inputField_2011.getText().trim();: Mengambil teks dari input field dan menghapus spasi di awal/akhir.

if (text_2011.isEmpty()) return;; Guard clause untuk mengabaikan input kosong.

String[] parts_2011 = text_2011.split(",");: Memecah string input berdasarkan pemisah koma menjadi array string.

array_2011 = new int[parts_2011.length];: Menginisialisasi array integer dengan ukuran sesuai jumlah elemen input.

Blok try-catch: Mencoba parse setiap string ke integer dengan Integer.parseInt(). Jika gagal (misal input huruf), NumberFormatException ditangkap dan JOptionPane.showMessageDialog menampilkan dialog error.

Metode setArrayFromInput_2011() - Inisialisasi State & UI

i_2011 = 1; stepCount_2011 = 1; sorting_2011 = true;; Reset variabel algoritma ke state awal dan aktifkan flag sorting.

stepButton_2011.setEnabled(true);: Aktifkan tombol "Langkah Selanjutnya" karena data sudah siap.

stepArea_2011.setText("");: Kosongkan area log untuk sesi baru.

panelArray_2011.removeAll();: Hapus semua komponen label lama dari panel visualisasi.

labelArray_2011 = new JLabel[array_2011.length];: Inisialisasi array referensi label sesuai jumlah data.

Metode setArrayFromInput_2011() - Rendering Label Array

Loop for untuk setiap elemen array:

labelArray_2011[k_2011] = new JLabel(String.valueOf(array_2011[k_2011]));: Buat label dengan teks nilai angka.

setFont(new Font("Arial", Font.BOLD, 24));: Atur font besar dan tebal agar visualisasi jelas.

`setBorder(BorderFactory.createLineBorder(Color.BLACK));` Tambahkan border hitam untuk membatasi area setiap label.

`setPreferredSize(new Dimension(50, 50));` Tetapkan ukuran tetap 50x50 pixel untuk konsistensi tampilan.

`setHorizontalAlignment(SwingConstants.CENTER);` Ratakan teks di tengah label.

`panelArray_2011.add(labelArray_2011[k_2011]);` Tambahkan label ke panel visualisasi.

`panelArray_2011.revalidate(); panelArray_2011.repaint();` Penting: Meminta layout manager menghitung ulang posisi komponen (`revalidate`) dan menggambar ulang panel (`repaint`) agar perubahan tampil di layar.

Metode `performStep_2011()` - Eksekusi Satu Langkah Algoritma

`if (i_2011 < array_2011.length && sorting_2011):` Guard clause: hanya eksekusi jika masih ada elemen yang belum diproses dan sorting sedang aktif.

`int key_2011 = array_2011[i_2011];` Ambil elemen kunci pada posisi `i` (elemen yang akan disisipkan).

`j_2011 = i_2011 - 1;` Inisialisasi pointer `j` ke elemen sebelum key.

`StringBuilder stepLog_2011 = new StringBuilder();` Siapkan buffer untuk mencatat log langkah ini.

Metode `performStep_2011()` - Logika Shift dan Insert

`while (j_2011 >= 0 && array_2011[j_2011] > key_2011):` Loop selama elemen di kiri key lebih besar dari key (kondisi untuk pengurutan ascending).

`array_2011[j_2011 + 1] = array_2011[j_2011];` Geser elemen yang lebih besar satu posisi ke kanan.

`j_2011--;` Gerakkan pointer `j` ke kiri untuk perbandingan berikutnya.

`array_2011[j_2011 + 1] = key_2011;` Insertion Step: Sisipkan key ke posisi yang tepat setelah elemen-elemen besar bergeser.

Catatan: Dalam satu pemanggilan `performStep_2011()`, seluruh proses shift untuk satu key dieksekusi sekaligus. Ini berbeda dengan visualisasi yang menampilkan setiap pergeseran individual.

Metode `performStep_2011()` - Update UI dan Log

`updateLabels_2011();` Panggil metode helper untuk memperbarui teks pada semua label sesuai nilai array terkini.

stepLog_2011.append("Hasil:
").append(arrayToString_2011(array_2011)).append("\n\n");: Tambahkan representasi string array ke log.

stepArea_2011.append(stepLog_2011.toString());: Tampilkan log ke area teks GUI.

i_2011++; stepCount_2011++;: Increment counter untuk iterasi dan penomoran langkah berikutnya.

if (i_2011 == array_2011.length): Cek apakah semua elemen sudah diproses.

sorting_2011 = false; stepButton_2011.setEnabled(false);: Nonaktifkan flag dan tombol karena sorting selesai.

JOptionPane.showMessageDialog(this, "Sorting Selesai!");: Tampilkan dialog konfirmasi penyelesaian.

Metode updateLabels_2011() - Sinkronisasi Data ke UI

Loop melalui semua index array:
labelArray_2011[k_2011].setText(String.valueOf(array_2011[k_2011]));

Memastikan tampilan visual (label) selalu mencerminkan nilai aktual dalam array data setelah setiap operasi modifikasi.

Metode reset_2011() - Reset Aplikasi ke State Awal

inputField_2011.setText("");: Kosongkan field input teks.

panelArray_2011.removeAll(); panelArray_2011.revalidate();

panelArray_2011.repaint();: Hapus semua label visualisasi dan refresh panel.

stepArea_2011.setText("");: Kosongkan area log.

stepButton_2011.setEnabled(false);: Nonaktifkan tombol step.

sorting_2011 = false; i_2011 = 1; stepCount_2011 = 1;: Reset semua variabel state ke nilai default.

Metode arrayToString_2011() - Helper Formatting

StringBuilder sb_2011 = new StringBuilder();: Gunakan StringBuilder untuk efisiensi concatenation string dalam loop.

Loop append setiap elemen array ke buffer.

if (k_2011 < arr_2011.length - 1) sb_2011.append(", ");: Tambahkan pemisah koma hanya di antara elemen, bukan setelah elemen terakhir.

`return sb_2011.toString();`;; Kembalikan string hasil formatting untuk ditampilkan di log.

Metode `main()` - Thread Safety dengan `SwingUtilities`

`SwingUtilities.invokeLater()` -> { ... };; Sangat Penting. Memastikan pembuatan dan penampilan GUI dilakukan di Event Dispatch Thread (EDT), thread khusus Swing untuk menangani event dan rendering UI.

Mencegah race condition dan masalah thread safety yang dapat menyebabkan GUI freeze atau perilaku tidak terduga.

`InsertionSortGUI_2511532011 gui_2011 = new InsertionSortGUI_2511532011();`
`gui_2011.setVisible(true);`;; Instansiasi frame dan tampilkan ke layar.

2.2 Tugas Pratikum

```
package pkeam7_2511532811;

import java.awt.EventQueue;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class SortingGUI_2511532811 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField fName_2011;
    private JTextField fNm_2011;
    private JTextField fFakultas_2011;
    private JTextArea DWHs_2011;
    private JTextArea SWhs_2011;
    private ArrayList<Mahasiswa_2011> listMahasiswa_2011 = new ArrayList<Mahasiswa_2011>();

    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    SortingGUI_2511532811 frame = new SortingGUI_2511532811();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    public SortingGUI_2511532811() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 450, 280);
        contentPane = new JPanel();
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);
        contentPane.setLayout(null);

        fName_2011 = new JTextField();
        fName_2011.setBounds(10, 10, 90, 20);
        contentPane.add(fName_2011);
        fName_2011.setColumns(10);

        fNm_2011 = new JTextField();
        fNm_2011.setBounds(10, 30, 90, 20);
        contentPane.add(fNm_2011);
        fNm_2011.setColumns(10);

        fFakultas_2011 = new JTextField();
        fFakultas_2011.setBounds(10, 50, 90, 20);
        contentPane.add(fFakultas_2011);
        fFakultas_2011.setColumns(10);

        JButton btnInput_2011 = new JButton("Input");
        btnInput_2011.setBounds(10, 70, 90, 23);
        contentPane.add(btnInput_2011);

        JButton btnSort_2011 = new JButton("Sorting");
        btnSort_2011.setBounds(10, 90, 90, 23);
        contentPane.add(btnSort_2011);

        JButton btnReset_2011 = new JButton("Reset");
        btnReset_2011.setBounds(10, 110, 90, 23);
        contentPane.add(btnReset_2011);

        JComboBox<String> DOSSorting_2011 = new JComboBox<String>();
        DOSSorting_2011.setBounds(10, 130, 90, 23);
        contentPane.add(DOSSorting_2011);

        DWHs_2011 = new JTextArea();
        DWHs_2011.setEditable(false);

        SWhs_2011 = new JTextArea();
        SWhs_2011.setEditable(false);

        JScrollPane scrollPaneDWHs_2011 = new JScrollPane(DWHs_2011);
        scrollPaneDWHs_2011.setBounds(10, 150, 430, 50);
        contentPane.add(scrollPaneDWHs_2011);

        JScrollPane scrollPaneSWhs_2011 = new JScrollPane(SWhs_2011);
        scrollPaneSWhs_2011.setBounds(10, 170, 430, 70);
        contentPane.add(scrollPaneSWhs_2011);

        setupComboBox_2011(DOSSorting_2011);
        setupActionListeners_2011(btnInput_2011, btnSort_2011, btnReset_2011, DOSSorting_2011);

        private void setupComboBox_2011(JComboBox<String> combo_2011) {
            combo_2011.addItem("Insertion Sort");
            combo_2011.addItem("Selection Sort");
            combo_2011.addItem("Bubble Sort");
        }

        private void setupActionListeners_2011(final JButton btnInput_2011, final JButton btnSort_2011,
            final JButton btnReset_2011, final JComboBox<String> combo_2011) {

            btnInput_2011.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e_2011) {
                    inputData_2011();
                }
            });

            btnSort_2011.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e_2011) {
                    startSorting_2011(combo_2011);
                }
            });

            btnReset_2011.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e_2011) {
                    resetData_2011();
                }
            });
        }

        private void inputData_2011() {
            String nama_2011 = fName_2011.getText().trim();
            String nm_2011 = fNm_2011.getText().trim();
            String fakultas_2011 = fFakultas_2011.getText().trim();

            if (nama_2011.isEmpty() || nm_2011.isEmpty() || fakultas_2011.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Semua field wajib diisi!", "Input Kosong", JOptionPane.WARNING_MESSAGE);
                return;
            }

            Mahasiswa_2011 m_2011 = new Mahasiswa_2011();
            m_2011.setNama_2011(nama_2011);
            m_2011.setNm_2011(nm_2011);
            m_2011.setFakultas_2011(fakultas_2011);

            listMahasiswa_2011.add(m_2011);
            updateDisplay_2011();

            fName_2011.setText("");
            fNm_2011.setText("");
            fFakultas_2011.setText("");
            fName_2011.requestFocus();
        }

        private void updateDisplay_2011() {
            StringBuilder sb_2011 = new StringBuilder("DATA MAHASISWA:\n");
            sb_2011.append("-----\n");
            for (int i_2011 = 0; i_2011 < listMahasiswa_2011.size(); i_2011++) {
                Mahasiswa_2011 m_2011 = listMahasiswa_2011.get(i_2011);
                sb_2011.append(String.format("%-20s %-10s | NIM: %-10s | %s\n",
                    i_2011 + 1, m_2011.getNama_2011(), m_2011.getNm_2011(), m_2011.getFakultas_2011()));
            }
            DWHs_2011.setText(sb_2011.toString());
        }

        private void startSorting_2011(final JComboBox<String> combo_2011) {
            if (listMahasiswa_2011.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Data masih kosong!", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }

            final String pilihan_2011 = (String) combo_2011.getSelectedItemId();
            SWhs_2011.setText("== MEMULAI SORTING ==\nData Awal: " + formatList_2011() + "\n\n");

            Thread thread_2011 = new Thread(new Runnable() {
                public void run() {
                    try {
                        if ("Insertion Sort".equals(pilihan_2011)) {
                            insertionSort_2011();
                        } else if ("Selection Sort".equals(pilihan_2011)) {
                            selectionSort_2011();
                        } else if ("Bubble Sort".equals(pilihan_2011)) {
                            bubbleSort_2011();
                        }
                        SwingUtilities.invokeLater(new Runnable() {
                            public void run() {
                                SWhs_2011.append("\n===== Selesai! ");
                            }
                        });
                    }
                }
            });
        }
    }
}
```



```

updateDisplay_2011();
        }
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        }
    });
    thread_2011.start();
}

private String formatList_2011() {
    StringBuilder sb_2011 = new StringBuilder("");
    for (int i_2011 = 0; i_2011 < listMahasiswa_2011.size(); i_2011++) {
        sb_2011.append(listMahasiswa_2011.get(i_2011).getNama_2011());
        if (i_2011 < listMahasiswa_2011.size() - 1) sb_2011.append(", ");
    }
    sb_2011.append("\n");
    return sb_2011.toString();
}

private void logStep_2011(final String msg_2011) throws InterruptedException {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            SMS_2011.append(msg_2011 + "\n");
            SMS_2011.setCaretPosition(SMS_2011.getDocument().getLength());
        }
    });
    Thread.sleep(100);
}

private void resetData_2011() {
    listMahasiswa_2011.clear();
    SMS_2011.setText("");
    FNama_2011.setText("");
    FNim_2011.setText("");
    FFakultas_2011.setText("");
}

private void insertionSort_2011() throws InterruptedException {
    logStep_2011(">> INSERTION SORT");
    for (int i_2011 = 1; i_2011 < listMahasiswa_2011.size(); i_2011++) {
        Mahasiswa_2011 key_2011 = listMahasiswa_2011.get(i_2011);
        int j_2011 = i_2011 - 1;
        while (j_2011 >= 0 && listMahasiswa_2011.get(j_2011).getNama_2011().compareToIgnoreCase(key_2011.getNama_2011()) > 0) {
            listMahasiswa_2011.set(j_2011 + 1, listMahasiswa_2011.get(j_2011));
            j_2011--;
        }
        listMahasiswa_2011.set(j_2011 + 1, key_2011);
        logStep_2011("Langkah " + i_2011 + " : " + formatList_2011());
    }
}

private void selectionSort_2011() throws InterruptedException {
    logStep_2011(">> SELECTION SORT");
    for (int i_2011 = 0; i_2011 < listMahasiswa_2011.size() - 1; i_2011++) {
        int minIdx_2011 = i_2011;
        for (int j_2011 = i_2011 + 1; j_2011 < listMahasiswa_2011.size(); j_2011++) {
            if (listMahasiswa_2011.get(j_2011).getNama_2011().compareToIgnoreCase(listMahasiswa_2011.get(minIdx_2011).getNama_2011()) < 0) {
                minIdx_2011 = j_2011;
            }
        }
        Mahasiswa_2011 temp_2011 = listMahasiswa_2011.get(i_2011);
        listMahasiswa_2011.set(i_2011, listMahasiswa_2011.get(minIdx_2011));
        listMahasiswa_2011.set(minIdx_2011, temp_2011);
        logStep_2011("Pass " + (i_2011 + 1) + " : " + formatList_2011());
    }
}

private void bubbleSort_2011() throws InterruptedException {
    logStep_2011(">> BUBBLE SORT");
    int n_2011 = listMahasiswa_2011.size();
    for (int i_2011 = 0; i_2011 < n_2011 - 1; i_2011++) {
        boolean swapped_2011 = false;
        for (int j_2011 = 0; j_2011 < n_2011 - i_2011 - 1; j_2011++) {
            if (listMahasiswa_2011.get(j_2011).getNama_2011().compareToIgnoreCase(listMahasiswa_2011.get(j_2011 + 1).getNama_2011()) > 0) {
                Mahasiswa_2011 temp_2011 = listMahasiswa_2011.get(j_2011);
                listMahasiswa_2011.set(j_2011, listMahasiswa_2011.get(j_2011 + 1));
                listMahasiswa_2011.set(j_2011 + 1, temp_2011);
                swapped_2011 = true;
            }
        }
        logStep_2011("Pass " + (i_2011 + 1) + " : " + formatList_2011());
        if (!swapped_2011) break;
    }
}

static class Mahasiswa_2011 {
    private String nama_2011;
    private String nim_2011;
    private String fakultas_2011;

    public Mahasiswa_2011() {
    }

    public Mahasiswa_2011(String nama_2011, String nim_2011, String fakultas_2011) {
        this.nama_2011 = nama_2011;
        this.nim_2011 = nim_2011;
        this.fakultas_2011 = fakultas_2011;
    }

    public String getNama_2011() {
        return nama_2011;
    }

    public void setNama_2011(String nama_2011) {
        this.nama_2011 = nama_2011;
    }

    public String getNim_2011() {
        return nim_2011;
    }

    public void setNim_2011(String nim_2011) {
        this.nim_2011 = nim_2011;
    }

    public String getFakultas_2011() {
        return fakultas_2011;
    }

    public void setFakultas_2011(String fakultas_2011) {
        this.fakultas_2011 = fakultas_2011;
    }
}
}

```

1. Konstruktor - Komponen Input Mahasiswa

FName_2011 = new JTextField(); FName_2011.setBounds(10, 11, 96, 20);: Field input nama di posisi (10,11) dengan ukuran 96x20 pixel.

FNim_2011 = new JTextField(); FNim_2011.setBounds(116, 11, 96, 20);: Field input NIM di sebelah kanan field nama.

FFakultas_2011 = new JTextField(); FFakultas_2011.setBounds(222, 11, 96, 20);: Field input fakultas di sebelah kanan field NIM.

Ketiga field disusun horizontal di bagian atas form.

2. Konstruktor - Tombol Aksi

```
JButton btnInput_2011 = new JButton("Input");  
btnInput_2011.setBounds(337, 10, 89, 23);: Tombol untuk menambahkan  
data mahasiswa ke list.
```

```
JButton btnSort_2011 = new JButton("Sorting");  
btnSort_2011.setBounds(235, 148, 89, 23);: Tombol untuk memulai  
proses pengurutan.
```

```
JButton btnReset_2011 = new JButton("Reset");  
btnReset_2011.setBounds(337, 148, 89, 23);: Tombol untuk  
mengosongkan semua data dan reset aplikasi.
```

3. Konstruktor - ComboBox Pemilihan Algoritma

```
JComboBox<String> DDSorting_2011 = new JComboBox<>();  
DDSorting_2011.setBounds(10, 148, 160, 22);: Dropdown untuk memilih  
algoritma sorting yang akan digunakan.  
setupComboBox_2011(DDSorting_2011);: Memanggil metode helper  
untuk mengisi opsi algoritma.
```

4. Konstruktor - TextArea dengan ScrollPane

```
DMhs_2011 = new JTextArea(); DMhs_2011.setEditable(false);: Area  
teks read-only untuk menampilkan daftar mahasiswa.
```

```
JScrollPane scrollPaneDMhs_2011 = new JScrollPane(DMhs_2011);:   
Membungkus JTextArea dengan scrollbar otomatis jika konten melebihi  
area tampil.
```

```
scrollPaneDMhs_2011.setBounds(10, 42, 416, 95);: Posisi dan ukuran  
panel scroll untuk daftar data.
```

Pola yang sama diterapkan untuk SMhs_2011 (area log sorting) di bagian bawah.

Metode setupComboBox_2011() - Inisialisasi Opsi

```
combo_2011.addItem("Insertion Sort");: Menambahkan opsi pertama ke  
dropdown.
```

```
combo_2011.addItem("Selection Sort");: Opsi kedua.
```

```
combo_2011.addItem("Bubble Sort");: Opsi ketiga.
```

Pengguna dapat memilih salah satu dari tiga algoritma klasik ini.

5. Metode inputData_2011() - Validasi dan Input

String nama_2011 = FName_2011.getText().trim();: Ambil dan bersihkan input dari field teks.

if (nama_2011.isEmpty() || ...): Validasi bahwa semua field wajib diisi.

JOptionPane.showMessageDialog(...): Tampilkan dialog warning jika validasi gagal.

Mahasiswa_2011 m_2011 = new Mahasiswa_2011();: Buat objek mahasiswa baru.

m_2011.setNama_2011(nama_2011);: Set nilai atribut menggunakan setter.

listMahasiswa_2011.add(m_2011);: Tambahkan objek ke ArrayList (model data).

updateDisplay_2011();: Refresh tampilan daftar mahasiswa.

FName_2011.setText(""); FName_2011.requestFocus();: Kosongkan form dan kembalikan fokus ke field nama untuk input berikutnya.

6. Metode updateDisplay_2011() - Render Data ke UI

StringBuilder sb_2011 = new StringBuilder("DATA MAHASISWA:\n");::

Gunakan StringBuilder untuk efisiensi concatenation string.

Loop melalui listMahasiswa_2011

7. Metode insertionSort_2011() - Implementasi Algoritma

logStep_2011(">> INSERTION SORT");: Catat judul algoritma di log.

for (int i_2011 = 1; i_2011 < listMahasiswa_2011.size(); i_2011++): Outer loop mulai dari index 1.

Mahasiswa_2011 key_2011 = listMahasiswa_2011.get(i_2011);: Ambil elemen kunci.

While (j_2011 >= 0 &&

listMahasiswa_2011.get(j_2011).getNama_2011().compareToIgnoreCase(key_2011.getNama_2011()) > 0):

String Comparison: Gunakan compareToIgnoreCase() untuk membandingkan nama secara leksikografis case-insensitive.

Geser elemen yang lebih besar (secara alfabet) ke kanan.

listMahasiswa_2011.set(j_2011 + 1, key_2011);: Sisipkan key di posisi yang tepat.

logStep_2011("Langkah " + i_2011 + ": " + formatList_2011());: Catat state array setelah setiap iterasi.

8. Metode selectionSort_2011() - Implementasi Algoritma

for (int i_2011 = 0; i_2011 < listMahasiswa_2011.size() - 1; i_2011++):

Outer loop untuk setiap posisi target.

int minIdx_2011 = i_2011;; Asumsi elemen di posisi i adalah minimum.

Inner loop cari elemen dengan nama terkecil (alfabet paling awal) di sisa array.

if

(listMahasiswa_2011.get(j_2011).getNama_2011().compareToIgnoreCase (...) < 0): Update minIdx_2011 jika ditemukan nama yang lebih kecil.

Swap: Tukar elemen di posisi i dengan elemen minimum yang ditemukan menggunakan variabel temporary.

logStep_2011("Pass " + (i_2011 + 1) + ": " + formatList_2011());: Catat state setelah setiap pass.

9. Metode bubbleSort_2011() - Implementasi Algoritma dengan Optimasi

int n_2011 = listMahasiswa_2011.size();: Simpan ukuran list.

for (int i_2011 = 0; i_2011 < n_2011 - 1; i_2011++): Outer loop untuk jumlah pass.

boolean swapped_2011 = false;; Flag optimasi. Catat apakah ada swap terjadi di pass ini.

Inner loop bandingkan dan tukar elemen bersebelahan jika urutannya terbalik.

if

(listMahasiswa_2011.get(j_2011).getNama_2011().compareToIgnoreCase (...) > 0): Jika elemen kiri > kanan, swap.

swapped_2011 = true;; Tandai bahwa swap terjadi.

if (!swapped_2011) break;; Optimasi Early Exit. Jika satu pass lengkap tanpa swap, berarti array sudah terurut, hentikan algoritma lebih awal.

logStep_2011("Pass " + (i_2011 + 1) + ": " + formatList_2011());: Catat state setiap pass.

10. Inner Class Mahasiswa_2011 - Model Data

static class Mahasiswa_2011: Kelas statis nested yang merepresentasikan entitas mahasiswa.

private String nama_2011, nim_2011, fakultas_2011;; Atribut privat untuk enkapsulasi data.

Konstruktor: Default constructor dan parameterized constructor untuk fleksibilitas instansiasi.

Getter/Setter: Metode publik untuk akses dan modifikasi atribut secara terkontrol, menerapkan prinsip enkapsulasi Java Bean.

BAB III KESIMPULAN

3.1 Kesimpulan

Berdasarkan hasil praktikum dan pembahasan, telah berhasil diimplementasikan tiga algoritma pengurutan dasar yaitu Insertion Sort, Selection Sort, dan Bubble Sort menggunakan Java secara in-place serta dibangun aplikasi GUI berbasis Java Swing untuk memvisualisasikan proses Insertion Sort secara interaktif langkah demi langkah. Praktikum ini menegaskan pemahaman mendalam mengenai kompleksitas waktu $O(n^2)$, logika perulangan bersarang, dan manipulasi array melalui variabel temporary, sekaligus menunjukkan kemampuan integrasi komponen GUI seperti JTextField, JLabel, dan JTextArea untuk menghubungkan logika algoritma backend dengan antarmuka pengguna, sehingga tujuan praktikum dalam memberikan pemahaman komprehensif mengenai algoritma pengurutan serta keterampilan teknis pembangunan aplikasi visualisasi tercapai secara optimal.

DAFTAR PUSTAKA

[1]W3schools,“JavaDataStructur”2026[Daring].Tersediapada:https://www.w3schools.com/java/java_data_structures.asp.[Diakses:20-Meil-2026]

